

Kolumnowe bazy danych cz. I

Eksploracja danych, podstawowe agregacje i pierwszy benchmark PostgreSQL i ClickHouse

Imię i nazwisko: Marek Małek, Mateusz Lampert

Grupa: 4 (Piątek, 15:00-16:30)

Cel ćwiczenia

- sprawdzenie poprawności działania przygotowanego środowiska,
- zapoznanie się z tabelą events,
- przećwiczenie podstawowych zapytań analitycznych SQL,
- przygotowanie pierwszych prostych wskaźników KPI,
- porównanie wyników uzyskanych w PostgreSQL i ClickHouse,
- sformułowanie pierwszych wniosków, w jakich analizach baza kolumnowa może być szczególnie użyteczna.

Ważne

- Pracuj na tej samej tabeli events w obu bazach danych.
- W typowej konfiguracji na zajęciach wystarczy używać nazwy events.
- Jeżeli w Twoim kliencie SQL tabela nie jest widoczna w bieżącym kontekście, użyj pełnej nazwy: public.events w PostgreSQL albo ds_lab.events w ClickHouse.
- Tam, gdzie polecenie mówi o porównaniu, nie wystarczy samo uruchomienie zapytania - napisz krótko, czy wyniki są zgodne.
- Do każdego zadania dołącz: kod zapytania, wynik oraz wymagany komentarz.
- Nie oddawaj samych zrzutów wyników bez interpretacji.
- W kilku zadaniach możesz korzystać z zapytań startowych, ale część rozwiązań musisz samodzielnie rozbudować.
- Do benchmarku użyj tego samego klienta SQL dla obu baz danych.
- Być może na zajęciach zabraknie czasu na wykonanie wszystkich zadań. Pozostałe elementy należy dokończyć po zajęciach.

Opis kolumn tabeli events

- event_time - czas zdarzenia,
- event_type - typ zdarzenia, np. view, cart, purchase,
- user_id - identyfikator użytkownika,
- session_id - identyfikator sesji,
- country - kraj,
- device - urządzenie,
- product_id - identyfikator produktu,
- price - cena jednostkowa,
- quantity - liczba sztuk.

Sprawozdanie

Oddawane sprawozdanie powinno zawierać komplet rozwiązań wszystkich zadań: kod zapytań, wyniki oraz wymagane komentarze.

- Sprawozdanie oddaj jako plik PDF albo Markdown.
- Kod SQL formatuj jako bloki kodu.
- Wyniki zapytań przedstawiaj jako tabele albo zrzuty ekranu.
- Komentarze pisz pełnymi zdaniami.
- Termin oddania: Do końca dnia poprzedzającego kolejne zajęcia

Punktacja: razem 10 pkt.

0. Gotowość do zajęć - 0 pkt

Pokaż, że środowisko jest przygotowane do pracy.

- docker compose up -d uruchamia oba kontenery bez błędów,
- docker ps pokazuje kontenery postgres i clickhouse w statusie Up,
- w obu bazach dostępna jest tabela events,
- możliwe jest połączenie z obiema bazami z poziomu klienta SQL.

Uwaga: brak gotowości środowiska uniemożliwia wykonanie dalszych zadań.

1. Pierwsze poznanie tabeli events - 1 pkt

Zapoznaj się z tabelą events w obu bazach.

- sprawdź strukturę tabeli,
- wyświetl kilka przykładowych rekordów,
- policz liczbę wierszy,
- wyznacz minimalny i maksymalny czas zdarzenia,
- sprawdź, czy w kolumnach price i quantity występują wartości NULL, jeśli występują, wskaż ich liczbę.

Zapytania startowe:

```
SELECT
    count(*) AS n,
    min(event_time) AS min_time,
    max(event_time) AS max_time
FROM events;

SELECT
    count(*) AS all_rows,
    count(price) AS non_null_price,
    count(quantity) AS non_null_quantity,
    count(*) - count(price) AS null_price_rows,
    count(*) - count(quantity) AS null_quantity_rows
FROM events;
```

W komentarzu napisz: czy liczba rekordów i zakres czasu są zgodne w obu bazach, czy w danych występują NULL-e.

Wyniki

Struktura tabeli

- PostgreSQL:

postgres=# \d events

Table "public.events"				
Column	Type	Collation	Nullable	Default
event_time	timestamp without time zone			
user_id	integer			
session_id	integer			
product_id	integer			
price	double precision			
quantity	integer			
country	text			
device	text			
event_type	text			

- ClickHouse:

```
fbf0d494de58 :) DESCRIBE events

DESCRIBE TABLE events

Query id: c1bc4fe2-8a6b-439c-8260-ab8b4cc1406d

1.  name      type      default_type  default_expression  comment  codec_expression  ttl_expression
2.  event_time DateTime
3.  user_id    UInt32
4.  session_id UInt32
5.  product_id UInt32
6.  price      Float64
7.  quantity   UInt32
8.  country    String
9.  device     String
   event_type String

9 rows in set. Elapsed: 0.002 sec.
```

Typy danych są zgodne co do nazwy, ale kolumny metadanych istotnie się różnią. PostgreSQL ma dodatkowo opcje collation (jak porównywać dane). Clickhouse oprócz wartości domyślnej ma też default_expression (jak zachowa się kolumna, gdy nie zostanie podana wartość), codecs_compression (jak kompresować dane) oraz ttl_expression (czy dane mają mieć czas życia).

Przykładowe rekordy

- PostgreSQL:

	event_time	user_id	session_id	product_id	price	quantity	country	device	event_ty
1	2025-05-05 03:56:41.000000	7297	6557	4507	126.22	1	DE	tablet	view
2	2025-04-16 21:35:32.000000	5698	154795	6913	20.73	1	IT	tablet	view
3	2025-04-19 23:35:29.000000	13032	187701	8929	212.66	1	ES	mobile	view
4	2025-02-01 00:03:58.000000	45754	110786	5575	142.55	1	US	mobile	add_to_car1
5	2025-01-19 18:43:51.000000	23527	90166	9892	135.94	1	SE	tablet	add_to_car1
6	2025-06-29 02:12:28.000000	24808	20657	9045	150.12	1	US	tablet	add_to_car1
7	2025-01-14 12:09:01.000000	3004	173347	3734	387.67	3	DE	desktop	view
8	2025-05-04 10:30:15.000000	23910	42639	6066	180.86	1	DE	tablet	view
9	2025-04-14 17:16:20.000000	47785	64176		233.82	1	NO	mobile	view

- ClickHouse:

Output ds_lab.events

	event_time	user_id	session_id	product_id	price	quantity	country	device	event_type
1	2025-06-01 00:00:02	14124	67477	4413	433.9	1	NL	mobile	view
2	2025-06-01 00:08:25	19541	128338	87	387.32	1	FR	mobile	view
3	2025-06-01 00:09:57	10339	4098	7979	44.96	1	DE	tablet	view
4	2025-06-01 00:13:24	13417	176048	1538	62.15	5	IT	tablet	add_to_cart
5	2025-06-01 00:14:57	27264	89143	2800	176.93	1	DE	desktop	view
6	2025-06-01 00:18:15	32892	70436	573	340.05	4	DE	desktop	view
7	2025-06-01 00:19:28	48533	69945	2341	164.19	1	FR	desktop	view
8	2025-06-01 00:19:49	15585	119659	5190	364.42	1	GB	mobile	view
9	2025-06-01 00:26:16	22648	56515			1	NL	mobile	view

1-500 of 501+

Wyniki są zgodne, oba systemy pokazują te same dane.

Liczba wierszy i zakres czasu

- PostgreSQL:

	n	min_time	max_time
1	100000	2025-01-01 00:05:42	2025-06-29 23:57:02

- ClickHouse:

	n	min_time	max_time
1	100000	2025-01-01 00:05:42.000000	2025-06-29 23:57:02.000000

Liczba wierszy jest zgodna w obu tabelach, podobnie jak zakres czasu zdarzeń. PostgreSQL stosuje dokładność do mikrosekund, a ClickHouse do sekund, ale oba zakresy są zgodne.

NULL-e w kolumnach price i quantity

- PostgreSQL:

Output

Result 10

CSV

	all_rows	non_null_price	non_null_quantity	null_price_rows	null_quantity_rows
1	100000	100000	100000	0	0

- ClickHouse:

Output						
Result 9						
all_rows ÷ non_null_price ÷ non_null_quantity ÷ null_price_rows ÷ null_quantity_rows ÷						
1	100000	100000	100000	0	0	

Wyniki są takie same, brak wartości NULL

2. Profil danych zdarzeniowych - 1 pkt

Przygotuj podstawowy profil danych.

- sprawdź, jakie typy zdarzeń występują w danych i ile ich jest,
- sprawdź, jakie kraje występują w danych i ile mają zdarzeń,
- sprawdź, jakie urządzenia występują w danych i ile mają zdarzeń,
- dla każdego z tych trzech przekrojów wskaź kategorię dominującą.

Zapytanie startowe:

```
SELECT
    event_type,
    count(*) AS n
FROM events
GROUP BY event_type
ORDER BY n DESC;
```

Dwa pozostałe zapytania przygotuj samodzielnie na analogicznej zasadzie.

Rozwiązanie:

```
SELECT
    event_type,
    count(*) AS n
FROM events
GROUP BY event_type
ORDER BY n DESC;
```

	event_type	n
1	view	79712
2	add_to_cart	15196
3	purchase	5092

```
SELECT
  country,
  count(*) AS n
FROM events
GROUP BY country
ORDER BY n DESC;
```

	country	n
1	GB	10212
2	US	10111
3	NO	10022
4	NL	9997
5	SE	9982
6	FR	9979
7	PL	9964
8	DE	9948
9	IT	9894
10	ES	9891

```
SELECT
    device,
    count(*) AS n
FROM events
GROUP BY device
ORDER BY n DESC;
```

	device	n
1	mobile	33392
2	tablet	33382
3	desktop	33226

Wśród wszystkich wydarzeń występują eventy 3 typów (view, add_to_cart oraz purchase), dominującą kategorią są zdarzenia typu view. Wydarzenia miały miejsce w 10 różnych krajach, z czego większość miała miejsce w Wielkiej Brytanii (GB). Wśród urządzeń dominowały telefony komórkowe, a wszystkie eventy dotyczyły jednego z typów urządzeń - mobile, tablet lub desktop.

3. Aktywność w czasie - 1 pkt

Sprawdź, jak zmienia się liczba zdarzeń w czasie.

- policz liczbę zdarzeń dla kolejnych dni,
- wskaż 5 dni o największej liczbie zdarzeń,
- wskaż 5 dni o najmniejszej liczbie zdarzeń,
- napisz, czy dane wyglądają na równomiernie rozłożone w czasie.

Możesz przygotować osobno: pełne zestawienie dzienne, 5 dni o największej liczbie zdarzeń oraz 5 dni o najmniejszej liczbie zdarzeń.

Wskazówka: w PostgreSQL możesz użyć DATE(event_time), a w ClickHouse możesz użyć toDate(event_time).

W komentarzu: napisz krótko, czy rozkład wygląda stabilnie, czy są widoczne dni odstające.

Rozwiązanie:

- Pełne zestawienie dzienne:

```
--- PostgreSQL
select date(event_time) as day, count(*) as event_count
from events
group by day;
```

	day	event_count
1	2025-06-15	562
2	2025-02-26	545
3	2025-06-16	509
4	2025-03-30	564
5	2025-06-13	538
6	2025-02-11	539
7	2025-03-28	559
8	2025-02-17	551
9	2025-01-09	544
10	2025-03-27	539
11	2025-06-18	562
180 rows		572


```
--- ClickHouse
select toDate(event_time) as day, count(*) as event_count
from events
group by day;
```

	day	event_count
1	2025-01-01	534
2	2025-01-02	563
3	2025-01-03	575
4	2025-01-04	534
5	2025-01-05	559
6	2025-01-06	562
7	2025-01-07	504
8	2025-01-08	549
9	2025-01-09	544
10	2025-01-10	550
11	2025-01-11	528
180 rows		569

- 5 dni o największej liczbie zdarzeń:

```
--- PostgreSQL
select date(event_time) as day, count(*) as event_count
from events
group by day
order by event_count desc
limit 5;
```

	📅 day 🗑	📅 event_count 🗑
1	2025-05-26	634
2	2025-03-12	613
3	2025-04-04	611
4	2025-03-10	610
5	2025-01-25	605

```
--- ClickHouse
select toDate(event_time) as day, count(*) as event_count
from events
group by day
order by event_count desc
limit 5;
```

	📅 day 🗑	📅 event_count 🗑
1	2025-05-26	634
2	2025-03-12	613
3	2025-04-04	611
4	2025-03-10	610
5	2025-01-25	605

- 5 dni o najmniejszej liczbie zdarzeń:

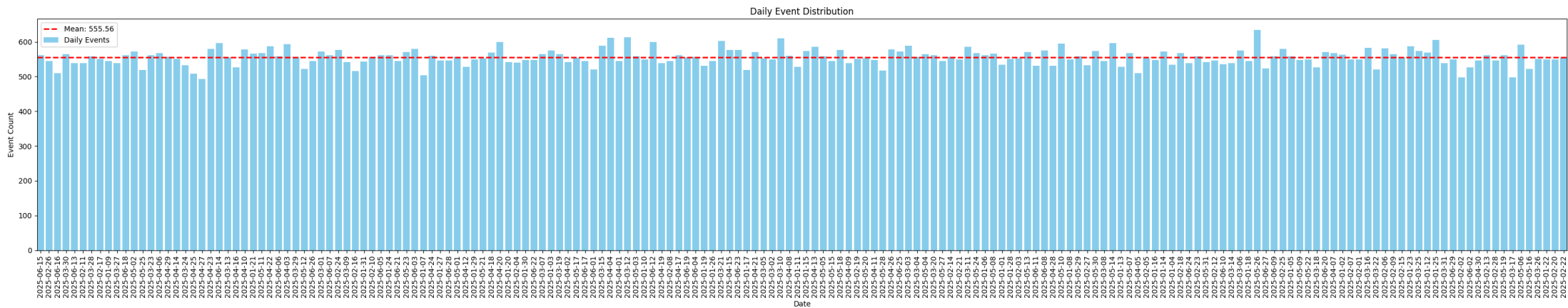
```
--- PostgreSQL
select date(event_time) as day, count(*) as event_count
from events
group by day
order by event_count asc
limit 5;
```

	day	event_count
1	2025-04-27	493
2	2025-02-02	497
3	2025-03-17	497
4	2025-01-07	504
5	2025-04-25	508

```
--- ClickHouse
select toDate(event_time) as day, count(*) as event_count
from events
group by day
order by event_count asc
limit 5;
```

	day	event_count
1	2025-04-27	493
2	2025-02-02	497
3	2025-03-17	497
4	2025-01-07	504
5	2025-04-25	508

- Rozkład liczby zdarzeń w czasie (histogram + średnia):



Komentarz:

- rozkład jest raczej stabilny, ale jest kilka dni odstających. Najbardziej odstaje 2025-05-26 (634 zdarzenia do 555 (średnia) ~14%).

4. Podstawowe KPI sprzedażowe - 2 pkt

Przygotuj podstawowe KPI związane ze zdarzeniami zakupowymi.

- policz liczbę zdarzeń typu purchase,
- policz łączny przychód ze zdarzeń typu purchase,
- policz średnią wartość pojedynczego zakupu,
- policz średnią liczbę sztuk w pojedynczym zakupie,
- policz liczbę sesji, w których wystąpił co najmniej jeden zakup,
- policz udział sesji zakupowych w ogólnej liczbie sesji jako uproszczony współczynnik konwersji.

Zapytanie startowe:

```
SELECT
    count(*) AS purchases_cnt,
    sum(price * quantity) AS revenue,
    avg(price * quantity) AS avg_order_value,
    avg(quantity) AS avg_quantity
FROM events
WHERE event_type = 'purchase';
```

Drugą część zadania - dotyczącą sesji i konwersji - przygotuj samodzielnie.

W komentarzu napisz: czy wyniki w obu bazach są zgodne oraz dlaczego udział sesji zakupowych lepiej opisuje konwersję niż prosty stosunek liczby zdarzeń purchase do liczby zdarzeń view.

Rozwiązanie:

```
-- postgres
with t as (
  select count(distinct session_id) as total_sessions
  from events
),
q as (
  select count(distinct session_id) as purchase_sessions
  from events
  where event_type = 'purchase'
)
select
  purchase_sessions,
  (purchase_sessions * 1.0) / total_sessions as conversion
from q, t;
```

10 ✓ ▾

select

11

uniqExactIf(session_id, event_type = 'purchase') as purchase_sessions,

12

purchase_sessions / count(distinct session_id) as conversion

13

from events;

Output

Result 11

×

1 row ▾

purchase_sessions

▽

▴

conversion

▽

▴

1

44601

0.2245747776960957

```
-- clickhouse
select
    uniqExactIf(session_id, event_type = 'purchase') as purchase_sessions,
    purchase_sessions / count(distinct session_id) as conversion
from events;
```

```
15 ✓ with t as (
16     select count(distinct session_id) as total_sessions
17     from events
18 ),
19 q as (
20     select count(distinct session_id) as purchase_sessions
21     from events
22     where event_type = 'purchase'
23 )
24 select
25     purchase_sessions,
26     (purchase_sessions * 1.0) / total_sessions as conversion
27 from q, t;
```

Output Result 13

1 row

	purchase_sessions	conversion
1	44601	0.22457477769609570901

Wyniki z obu zapytań zwróciły identyczne rezultaty (z dokładnością do dokładności numerycznej).

Komentarz:

- udział sesji zakupowych lepiej opisuje konwersję niż prosty stosunek liczby zdarzeń purchase do liczby zdarzeń view, ponieważ wiele zdarzeń purchase oraz view mogło wystąpić w trakcie jednej sesji. Konwersja obliczana w taki sposób (jako udział sesji zakupowych) pozwala na bardziej rzetelną ocenę, ponieważ nie bierze pod uwagę zachowań specyficznych dla poszczególnych użytkowników (jeden użytkownik może otwierać bardzo wiele zakładek i kupić tylko jeden przedmiot, a inny użytkownik może otworzyć tylko jeden link i kupić dany przedmiot) - dzięki temu nasza miara konwersji faktycznie odpowiada na pytanie ile razy, gdy ktoś wszedł do naszego sklepu, wizyta zakończyła się sprzedażą.

5. KPI w przekrojach biznesowych - 1 pkt

Policz przychód dla zdarzeń typu purchase w dwóch wybranych przekrojach:

- według kraju,
- według urządzenia,
- według dnia.

Wybierz dwa z powyższych przekrojów. Dla każdego policz przychód, posortuj wynik malejąco, wskaż najwyższe wartości i krótko je zinterpretuj.

Zapytanie startowe:

```
SELECT
    country,
    sum(price * quantity) AS revenue
FROM events
WHERE event_type = 'purchase'
GROUP BY country
ORDER BY revenue DESC;
```

Drugie zapytanie przygotuj samodzielnie.

Rozwiązanie

- według urządzenia (dla obu serwerów kod zapytania jest taki sam):

```
--- PostgreSQL & ClickHouse
select device,
       sum(price * quantity) as revenue
from events
where event_type = 'purchase'
group by device
order by revenue desc;
```

Wyniki:

- dla przekroju według kraju:
 - ClickHouse:

	country	revenue
1	GB	189364.49
2	IT	187879.44
3	NL	185101.92999999996
4	PL	173735.24999999997
5	FR	172757.88
6	NO	171390.430000000002
7	US	167135.53
8	ES	164466.48
9	DE	162931.26
10	SE	159174.65

• Postgres:

	country	revenue
1	GB	189364.489999999993
2	IT	187879.440000000018
3	NL	185101.929999999976
4	PL	173735.249999999997
5	FR	172757.879999999998
6	NO	171390.429999999993
7	US	167135.530000000006
8	ES	164466.479999999998
9	DE	162931.260000000007
10	SE	159174.649999999999

Wyniki są zgodne co do precyzji obliczeń.

- dla przekroju według urządzenia:
 - ClickHouse:

	device	revenue
1	mobile	590612.6200000001
2	tablet	573983.8700000001
3	desktop	569340.8500000001

- Postgres:

	device	revenue
1	mobile	590612.6200000003
2	tablet	573983.87
3	desktop	569340.8500000006

Wyniki są zgodne co do precyzji obliczeń.

Komentarz:

- Największy przychód pochodzi z Wielkiej Brytanii (country = ‘GB’), co może oznaczać, że jest to główny rynek. Jeśli chodzi o przekrój według urządzenia, to największy przychód pochodzi z urządzeń typu mobile, co może sugerować, że większość klientów korzysta z telefonów komórkowych do dokonywania zakupów (raczej popularny trend).

6. Użytkownicy o najwyższym przychodzie - 1 pkt

Znajdź użytkowników o najwyższym łącznym przychodzie z zakupów.

Wynik powinien zawierać: user_id, liczbę wszystkich zdarzeń użytkownika, liczbę zdarzeń typu purchase, łączny przychód użytkownika ze zdarzeń typu purchase. Pokaż co najmniej 20 rekordów o najwyższym przychodzie.

Ważne: Przychód licz tylko dla zdarzeń typu purchase.

Wskazówka: W tym zadaniu przydatne może być warunkowe zliczanie i warunkowe sumowanie.

Zapytanie startowe do uzupełnienia:

```
SELECT
    user_id,
    count(*) AS all_events,
    ... AS purchase_events,
    ... AS revenue
FROM events
GROUP BY user_id
ORDER BY revenue DESC
LIMIT 20;
```

W komentarzu napisz:

- czy użytkownik z najwyższym przychodem ma też największą liczbę wszystkich zdarzeń,
- czy duża aktywność użytkownika zawsze oznacza wysoki przychód,
- jakie wnioski można wyciągnąć z porównania liczby zdarzeń i przychodu.

Rozwiązanie:

```
-- postgres
select
    user_id,
    count(*) as all_events,
    count(case when event_type = 'purchase' then 1 end) as purchase_events,
    sum(case when event_type = 'purchase' then quantity * price else 0 end) as revenue
from events
group by user_id
order by revenue desc
limit 20;
```

	🔍 user_id 🔍	🔍 all_events 🔍	🔍 purchase_events 🔍	🔍 revenue 🔍
1	23810	3	2	3286.98000000000005
2	34269	6	2	2774.81
3	33378	5	2	2564.93
4	45959	3	2	2512.83
5	34262	2	1	2465.14999999999996
6	16444	3	1	2445.65
7	47090	3	1	2442.85
8	35823	2	1	2432.75
9	3940	4	1	2427.8
10	13037	2	1	2402.35000000000004
11	14089	3	1	2378.4
12	25552	2	1	2352.35000000000004
13	35735	3	1	2349.5
14	26366	1	1	2335.8
15	3042	7	1	2334.1
16	15277	2	1	2332.64999999999996
17	32253	5	3	2329.66
18	11051	2	1	2329
19	1189	3	1	2288
20	22320	5	1	2269.65

```
-- clickhouse
select
  user_id,
  count(*) as all_events,
  countIf(event_type = 'purchase') as purchase_events,
  sumIf(price * quantity, event_type = 'purchase') as revenue
from events
group by user_id
order by revenue desc
limit 20;
```

	🔍 user_id 🔍	🔍 all_events 🔍	🔍 purchase_events 🔍	🔍 revenue 🔍
1	23810	3	2	3286.98000000000005
2	34269	6	2	2774.81
3	33378	5	2	2564.93
4	45959	3	2	2512.83
5	34262	2	1	2465.14999999999996
6	16444	3	1	2445.65
7	47090	3	1	2442.85
8	35823	2	1	2432.75
9	3940	4	1	2427.8
10	13037	2	1	2402.35000000000004
11	14089	3	1	2378.4
12	25552	2	1	2352.35000000000004
13	35735	3	1	2349.5
14	26366	1	1	2335.8
15	3042	7	1	2334.1
16	15277	2	1	2332.64999999999996
17	32253	5	3	2329.66
18	11051	2	1	2329
19	1189	3	1	2288
20	22320	5	1	2269.65

Rezultaty obu zapytań są identyczne (z dokładnością do dokładności numerycznej):

Komentarz:

- użytkownik z najwyższym przychodem nie ma największej liczby wszystkich zdarzeń, nie ma też największej liczby zdarzeń typu purchase
- duża aktywność użytkownika pewnie w pewien sposób koreluje z wysokim przychodem, ale nie zawsze jednoznacznie oznacza wysoki przychód (aktywność oraz ilość wydarzeń typu purchase nie różni się w jakkolwiek znaczący sposób wśród użytkowników z miejsc 1-200)
- kluczowa dla wyniku jest faktyczna wartość zamówienia, a nie sama częstotliwość interakcji - użytkownik o niższej aktywności może wygenerować większy przychód jeśli kupuje droższe produkty.

7. Benchmark zapytań w PostgreSQL i ClickHouse - 3 pkt

W tym zadaniu wykonasz prosty benchmark zapytań w PostgreSQL i ClickHouse, aby porównać działanie obu systemów dla zapytań prostszych i bardziej złożonych agregacyjnie. Szczegóły zostały opisane w częściach A, B i C.

Sposób pomiaru

Dla każdego benchmarkowanego zapytania:

- uruchom zapytanie w PostgreSQL i w ClickHouse,
- wykonaj każde zapytanie 4 razy w każdej bazie,
- pierwszy pomiar pomiń jako rozgrzewkowy,
- zapisz czasy z trzech kolejnych uruchomień,
- podaj średni czas wykonania dla każdej bazy.

Czas odczytaj z klienta SQL, którego używasz na zajęciach.

Uwaga: nie benchmarkuj zapytań zwracających bardzo duży wynik bez LIMIT, ponieważ czas prezentacji danych w kliencie SQL może zaburzać porównanie.

Prezentacja wyników

Wyniki benchmarku przedstaw w zwartej tabeli. Tabela powinna zawierać co najmniej: nazwę zapytania, PostgreSQL - pomiar 1, pomiar 2, pomiar 3, średnia, oraz ClickHouse - pomiar 1, pomiar 2, pomiar 3, średnia.

Komentarz końcowy

Na końcu napisz 3-5 zdań komentarza, w których odniesiesz się do następujących kwestii:

- czy wyniki liczbowe zapytań były zgodne w obu bazach?
- czy różnice czasu wykonania były małe czy wyraźne?
- przy którym typie zapytania różnica była największa?
- czy po tym ćwiczeniu można postawić wstępny wniosek, że bardziej złożone agregacje są naturalnym zastosowaniem systemu analitycznego takiego jak ClickHouse?

Uwaga: nie oczekujemy jeszcze pełnego benchmarku technicznego. Celem zadania jest pierwsze praktyczne porównanie działania prostszych i bardziej złożonych zapytań agregujących w dwóch systemach.

Część A. Dwa zapytania z wcześniejszych zadań

Wybierz dwa zapytania z wcześniejszych zadań tego laboratorium i wykonaj je w obu bazach danych. Dla każdego zapytania pokaż wynik, napisz, czy wynik liczbowy jest zgodny w PostgreSQL i ClickHouse, oraz zapisz czas wykonania odczytany z klienta SQL.

Wybierz zapytania o różnym poziomie trudności, np. jedno prostsze zapytanie i jedno średnio złożone zapytanie.

Zapytanie z zadania 4

```
-- postgres
with t as (
    select count(distinct session_id) as total_sessions
    from events
),
q as (
    select count(distinct session_id) as purchase_sessions
    from events
    where event_type = 'purchase'
)
select
    purchase_sessions,
    (purchase_sessions * 1.0) / total_sessions as conversion
from q, t;
-- clickhouse
select
    uniqExactIf(session_id, event_type = 'purchase') as purchase_sessions,
    purchase_sessions / count(distinct session_id) as conversion
from events;
```

Zgodnie z wynikami z zadania 4, rezultaty obu zapytań są identyczne (z dokładnością do dokładności numerycznej).

Czasy wykonania zapytania:

	Postgres	Clickhouse
Time 1 [ms]	647	351
Time 2 [ms]	645	348
Time 3 [ms]	649	356
Average [ms]	647	351.67

Zapytanie w Clickhousie okazało się ponad 1.8 raza szybsze w porównaniu do Postgresa.

Zapytanie z zadania 6

```
-- postgres
select
    user_id,
    count(*) as all_events,
    count(case when event_type = 'purchase' then 1 end) as purchase_events,
    sum(case when event_type = 'purchase' then quantity * price else 0 end) as revenue
from events
group by user_id
order by revenue desc
limit 20;
-- clickhouse
select
    user_id,
    count(*) as all_events,
    countIf(event_type = 'purchase') as purchase_events,
    sumIf(price * quantity, event_type = 'purchase') as revenue
from events
group by user_id
order by revenue desc
limit 20;
```

Zgodnie z wynikami z zadania 6, rezultaty obu zapytań są identyczne (z dokładnością do dokładności numerycznej).

Czasy wykonania zapytania:

	Postgres	Clickhouse
Time 1 [ms]	585	375
Time 2 [ms]	553	355
Time 3 [ms]	560	352
Average [ms]	566	360.67

Zapytanie w Clickhousie okazało się ponad 1.5 razy szybsze w porównaniu do Postgresa.

Część B. Zapytanie benchmarkowe podane przez prowadzącego

Wykonaj w obu bazach poniższe zapytanie agregujące.

PostgreSQL:

```
SELECT
    DATE(event_time) AS day,
    country,
    device,
    event_type,
    count(*) AS events_cnt,
    count(DISTINCT user_id) AS users_cnt,
    count(DISTINCT session_id) AS sessions_cnt,
    sum(CASE
        WHEN event_type = 'purchase' THEN price * quantity
        ELSE 0
    END) AS revenue
FROM events
GROUP BY
    DATE(event_time),
    country,
    device,
    event_type
ORDER BY revenue DESC
LIMIT 20;
```

ClickHouse:

```
SELECT
    toDate(event_time) AS day,
    country,
    device,
    event_type,
    count() AS events_cnt,
    count(DISTINCT user_id) AS users_cnt,
    count(DISTINCT session_id) AS sessions_cnt,
    sum(CASE
        WHEN event_type = 'purchase' THEN price * quantity
        ELSE 0
    END) AS revenue
FROM events
GROUP BY
    day,
    country,
    device,
    event_type
ORDER BY revenue DESC
LIMIT 20;
```

Dla tego zapytania pokaż wynik z obu baz, napisz, czy wyniki są zgodne, oraz zapisz czas wykonania w PostgreSQL i ClickHouse.

Wyniki:

- Clickhouse:

	day	country	device	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-13	US	mobile	purchase	14	14	14	10827.56
2	2025-04-21	US	tablet	purchase	21	21	21	9945.939999999999
3	2025-01-23	IT	tablet	purchase	19	19	19	9584.16
4	2025-05-15	IT	desktop	purchase	15	15	15	9468.630000000001
5	2025-05-30	DE	desktop	purchase	18	18	18	9445.29
6	2025-03-04	US	tablet	purchase	18	18	18	9403.070000000002
7	2025-01-04	GB	tablet	purchase	20	20	20	9342.12
8	2025-02-04	FR	tablet	purchase	21	21	21	9139.39
9	2025-02-24	IT	desktop	purchase	16	16	16	9022.180000000002
10	2025-04-19	PL	mobile	purchase	15	15	15	8954.640000000001
11	2025-05-25	DE	mobile	purchase	15	15	15	8909.98
12	2025-06-10	DE	desktop	purchase	21	21	21	8884.72
13	2025-03-24	IT	desktop	purchase	17	17	17	8801.78
14	2025-03-20	IT	desktop	purchase	11	11	11	8796.710000000001
15	2025-05-10	IT	tablet	purchase	15	15	15	8755.74
16	2025-04-19	ES	mobile	purchase	13	13	13	8738.15
17	2025-06-14	PL	tablet	purchase	14	14	14	8689.470000000001
18	2025-04-26	US	tablet	purchase	14	14	14	8663.77
19	2025-03-25	SE	mobile	purchase	15	15	15	8570.77
20	2025-04-18	FR	mobile	purchase	13	13	13	8570.720000000001

- Postgres:

	day	country	device	event_type	events_cnt	users_cnt	sessions_cnt	revenue
1	2025-06-13	US	mobile	purchase	14	14	14	10827.559999999998
2	2025-04-21	US	tablet	purchase	21	21	21	9945.939999999997
3	2025-01-23	IT	tablet	purchase	19	19	19	9584.159999999998
4	2025-05-15	IT	desktop	purchase	15	15	15	9468.63
5	2025-05-30	DE	desktop	purchase	18	18	18	9445.289999999999
6	2025-03-04	US	tablet	purchase	18	18	18	9403.07
7	2025-01-04	GB	tablet	purchase	20	20	20	9342.119999999999
8	2025-02-04	FR	tablet	purchase	21	21	21	9139.390000000003
9	2025-02-24	IT	desktop	purchase	16	16	16	9022.180000000002
10	2025-04-19	PL	mobile	purchase	15	15	15	8954.64
11	2025-05-25	DE	mobile	purchase	15	15	15	8909.98
12	2025-06-10	DE	desktop	purchase	21	21	21	8884.72
13	2025-03-24	IT	desktop	purchase	17	17	17	8801.78
14	2025-03-20	IT	desktop	purchase	11	11	11	8796.71
15	2025-05-10	IT	tablet	purchase	15	15	15	8755.74
16	2025-04-19	ES	mobile	purchase	13	13	13	8738.15
17	2025-06-14	PL	tablet	purchase	14	14	14	8689.470000000001
18	2025-04-26	US	tablet	purchase	14	14	14	8663.769999999999
19	2025-03-25	SE	mobile	purchase	15	15	15	8570.769999999999
20	2025-04-18	FR	mobile	purchase	13	13	13	8570.720000000001

Rezultaty obu zapytań są identyczne (z dokładnością do dokładności numerycznej).

Czasy wykonania zapytania:

	Postgres	Clickhouse
Time 1 [ms]	1568	422
Time 2 [ms]	1603	425
Time 3 [ms]	1445	391
Average [ms]	1538.67	412.67

Średni czas zapytania w bazie Clickhouse jest prawie 4 razy krótszy w porównaniu do analogicznego zapytania w Postgresie.

Część C. Własne analogiczne zapytanie

Na podstawie zapytania z części B przygotuj jedno własne zapytanie analogiczne, które będzie oparte na tym samym schemacie, ale zmodyfikowane w co najmniej jednym lub dwóch elementach.

Możesz skorzystać z poniższych modyfikacji:

- ograniczyć dane tylko do zdarzeń purchase,
- usunąć jeden wymiar grupowania,
- dodać filtr dla wybranego kraju,
- dodać filtr dla wybranego urządzenia,
- zmienić zestaw liczonych miar,
- ograniczyć analizę do wybranego przedziału czasu.

Własne zapytanie wykonaj w obu bazach danych. Dla tego zapytania pokaż kod, pokaż wynik, napisz, czy wyniki są zgodne, oraz zapisz czas wykonania w PostgreSQL i ClickHouse.

Zapytanie:

- zdarzenia typu purchase,
- grupowanie tylko według dnia i kraju,
- filtr dla urządzenia mobile,
- analiza dla pierwszych 6 miesięcy 2025 roku
- filtr dla krajów “GB”, “US” i “DE”.

```
--- PostgreSQL
select
    date(event_time) as day,
    country,
    count() as purchase_cnt,
    sum(price * quantity) as total_revenue,
    avg(price * quantity) as avg_order_value
from events
where event_type = 'purchase'
    and device = 'mobile'
    and event_time >= '2025-01-01 00:00:00'
    and event_time < '2025-07-01 00:00:00'
    and country in ('GB', 'US', 'DE')
group by
    day,
    country
order by total_revenue desc
limit 20;

--- ClickHouse
select
    toDate(event_time) as day,
    country,
    count() as purchase_cnt,
    sum(price * quantity) as total_revenue,
    avg(price * quantity) as avg_order_value
from events
where event_type = 'purchase'
    and device = 'mobile'
    and event_time >= '2025-01-01 00:00:00'
    and event_time < '2025-07-01 00:00:00'
    and country in ('GB', 'US', 'DE')
group by
    day,
    country
order by total_revenue desc
limit 20;
```

Wyniki:

- Clickhouse:

	day	country	purchase_cnt	total_revenue	avg_order_value
1	2025-05-04	US	4	2904.6000000000004	726.1500000000001
2	2025-05-19	DE	3	2062.5099999999998	687.5033333333332
3	2025-06-27	DE	2	2057.67	1028.835
4	2025-06-12	GB	1	2021.9	2021.9
5	2025-02-02	GB	2	1969.86	984.93
6	2025-06-25	GB	4	1900.5600000000002	475.14000000000004
7	2025-01-29	GB	2	1808.6699999999998	904.3349999999999
8	2025-04-15	GB	1	1731.6499999999999	1731.6499999999999
9	2025-01-10	DE	1	1715.88	1715.88
10	2025-06-04	DE	2	1667.8999999999999	833.9499999999999
11	2025-01-31	DE	1	1621.04	1621.04
12	2025-05-17	GB	4	1614.44	403.61
13	2025-05-28	GB	2	1612.74	806.37
14	2025-05-16	DE	4	1609.4500000000003	402.36250000000007
15	2025-01-02	US	1	1591.36	1591.36
16	2025-05-06	US	6	1589.6599999999999	264.94333333333333
17	2025-04-19	GB	5	1585.94	317.188
18	2025-05-29	DE	2	1521.35	760.675
19	2025-01-07	US	2	1492.07	746.035
20	2025-02-01	DE	2	1479.0800000000002	739.5400000000001

- Postgres:

	📅 day 📄	🌐 country 📄	📄 purchase_cnt 📄	📄 total_revenue 📄	📄 avg_order_value 📄
1	2025-05-04	US	4	2904.6000000000004	726.1500000000001
2	2025-05-19	DE	3	2062.5099999999998	687.5033333333332
3	2025-06-27	DE	2	2057.67	1028.835
4	2025-06-12	GB	1	2021.9	2021.9
5	2025-02-02	GB	2	1969.86	984.93
6	2025-06-25	GB	4	1900.56	475.14
7	2025-01-29	GB	2	1808.6699999999998	904.3349999999999
8	2025-04-15	GB	1	1731.6499999999999	1731.6499999999999
9	2025-01-10	DE	1	1715.88	1715.88
10	2025-06-04	DE	2	1667.8999999999999	833.9499999999999
11	2025-01-31	DE	1	1621.04	1621.04
12	2025-05-17	GB	4	1614.44	403.61
13	2025-05-28	GB	2	1612.74	806.37
14	2025-05-16	DE	4	1609.45	402.3625
15	2025-01-02	US	1	1591.36	1591.36
16	2025-05-06	US	6	1589.6599999999999	264.9433333333333
17	2025-04-19	GB	5	1585.94	317.188
18	2025-05-29	DE	2	1521.35	760.675
19	2025-01-07	US	2	1492.07	746.035
20	2025-02-01	DE	2	1479.0800000000002	739.5400000000001

Wyniki są zgodne co do precyzji obliczeń.

	Postgres	Clickhouse
Time 1 [ms]	447	334
Time 2 [ms]	452	345
Time 3 [ms]	445	349
Average [ms]	448	342.67

Czas wykonania zapytania w Clickhousie jest około 1.3 razy krótszy w porównaniu do Postgresa.

Uwaga końcowa dla studentów

Oceniane będą:

- poprawność logiczna zapytania,
- zgodność wyników między bazami tam, gdzie jest to wymagane,
- umiejętność interpretacji wyniku,
- czytelność i sensowność kodu.

Samo uruchomienie gotowego zapytania bez zrozumienia i bez komentarza nie będzie traktowane jako pełne rozwiązanie.